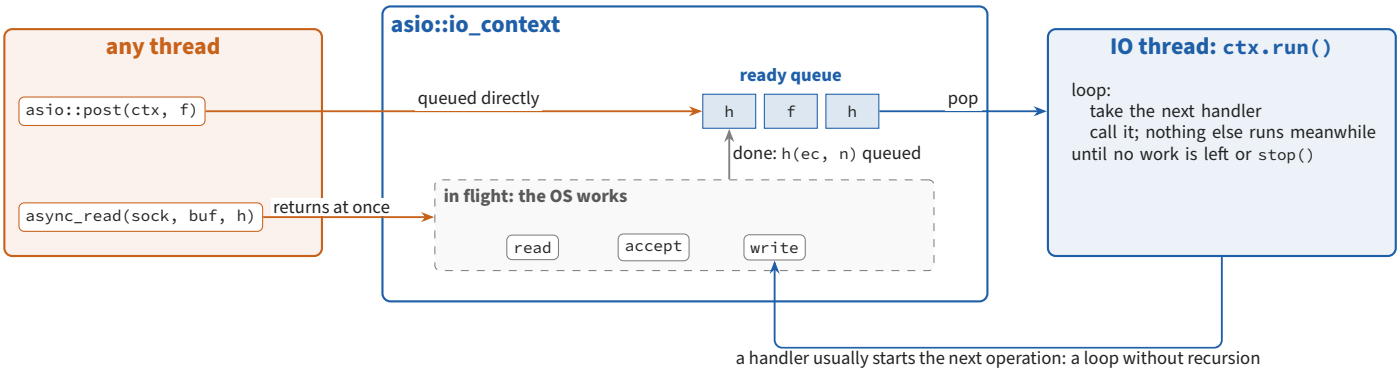
Asio cheat sheet

Thread colours used throughout: caller / app thread · IO thread (runs `run()`) · server thread · strand

standalone Asio 1.36 · as used in `src/net`

The whole idea in one picture



Asio is a **queue of callbacks**. You *start* an operation and hand it a *completion handler*. When the OS is done, the handler is queued, and it runs only when a thread is inside `run()`. In Tengen every `io_context` has **exactly one** such thread, owned by `TcpServer` or `TcpClient`.

Eight rules that follow

1. **Starting returns at once.** The handler runs later, never inside the `async_*` call, even if the data was already there. So `startRead()` calling itself from its handler is a loop, not recursion.
2. **Handlers run only inside `run()`.** One thread in `run()` ⇒ handlers never overlap: an *implicit strand*, for free.
3. **`run()` returns when it runs out of work.** A pending operation or a `work_guard` counts as work. After `stop()` queued handlers are *not* run; they die uncalled with the context.
4. **The handler must keep alive what it uses.** `asio::buffer()` is only pointer + size. Capture `self = shared_from_this()` and `shared_ptrs` to buffers in the lambda.
5. **Sockets aren't thread-safe.** A socket, acceptor or resolver is used by one thread at a time: the IO thread. Other threads `post()` the work to it. Posting itself is thread-safe.
6. **One `async_write` per socket at a time.** It is a loop of partial writes; a second one would interleave bytes. Hence a write queue. Same for reads.
7. **Cancel is not undo.** `close()` and `cancel()` make pending handlers run *anyway*, with `error::operation_aborted`. One that finished just before is queued with success. Each handler checks `ec` and "am I still current?" first.
8. **Errors are values.** Async handlers get an `error_code`; sync calls throw unless you pass one: `sock.close(ec)`. An exception leaving a handler leaves `run()` ⇒ `std::terminate` on a `std::thread`.

The two questions for every handler

Who keeps it alive? Every object, buffer and `this` it touches must outlive it (rule 4).
Is it still wanted? It may run after a cancel, a close or a reconnect (rule 7).

An IO thread's life

```
asio::io_context ctx;
auto guard = asio::make_work_guard(ctx); // keeps run() alive
std::thread io([&] { ctx.run(); });      // the IO thread

asio::post(ctx, [] { /* on io */ });     // from any thread

guard.reset(); // run() may end once the work is done ...
ctx.stop();    // ... or end now: queued ones not called
io.join();     // never from io itself: throws, terminates
```

Handler signatures

operation	handler
`async_read, async_write`	`(ec, size_t bytes)`
`acceptor.async_accept`	`(ec, tcp::socket)`
`resolver.async_resolve`	`(ec, results)`
`async_connect(sock, results)`	`(ec, endpoint)`
`post(ex, f)`	`()`

`ec` is an `asio::error_code`. The handler runs on error too; test `if (ec)` first. Only a stopped context never calls it.

Review checklist

1. Each `async_*`: who keeps object and buffer alive?
2. Does each handler check `ec` and "still current?" first?
3. Is a socket, acceptor or resolver touched off the IO thread?
4. At most one write and one read in flight per socket?
5. Is a callback called under a lock it could take again?
6. Can a handler wait for, join or destroy its own thread?
7. Can an exception leave a handler?
8. On stop: what is flushed, what dropped? Intended?

The calls used in `src/net`

call	what it does	keep in mind
Event loop		
`make_work_guard(ctx), .reset()`	counts as pending work	keeps an idle `run()` alive
`io_context::run()`	runs handlers on this thread until out of work	blocks; one thread per context here
`post(ctx or strand, f)`	queues `f` to run there	never runs inside `post`; any thread
`io_context::stop()`	makes `run()` return as soon as possible	any thread; queued handlers aren't called
Executors and strands		
`sock.get_executor()`	the context an object belongs to	what you build a strand from
`strand<any_io_executor> s(ex)`	sub-queue: never overlaps, keeps order	matters only with several `run()` threads
`bind_executor(s, h)`	the completion handler `h` runs through `s`	also the inner steps of `async_read/write`
TCP setup		
`acceptor.open / bind / listen`	server: a listening socket, step by step	`ec` overloads: no throws in the ctor
`resolver.async_resolve(host, port, h)`	client: name lookup	`cancel()` ⇒ `operation_aborted`
`async_connect(sock, results, h)`	client: tries each endpoint until one connects	`sock.async_connect` takes just one
`acceptor.async_accept(h)`	server: takes *one* waiting client	re-arm in the handler for the next
Data		
`buffer(ptr, n)`	a view on memory you own	valid until the handler runs
`array<const_buffer, 2>{a, b}`	a gather list: sent back to back	header + payload in one write
`async_read(sock, buf, h)`	reads *exactly* `buf.size()` bytes	peer closed ⇒ `error::eof`
`async_write(sock, bufs, h)`	writes everything	one at a time per socket (rule 6)
Closing		
`sock.shutdown(shutdown_both, ec)`	sends FIN; no more sends or receives	the peer's read ends with `eof`
`sock.close(ec)`	frees the socket, cancels its operations	handlers get `operation_aborted`
`resolver.cancel()`	aborts a running lookup	handler gets `operation_aborted`

Not used here, but you will meet them: `sock.async_read_some` (whatever has arrived) · `dispatch(ex, f)` (inline if already on `ex`, else posts) · `steady_timer` (timeouts) · `co_spawn` + `use_awaitable` (the same chains as coroutines).

What serializes what

tool	guarantees	in Tengen
one `run()` thread	handlers of that context never overlap	all of TcpServer, TcpClient
strand	its handlers never overlap and run in post order	`Connection` write queue
`post`	the work runs on the thread that owns the object	`send`, `close`, cancelling
`std::mutex`	state shared with threads outside asio	connection map, client state
queue + thread	work leaves the IO thread, in order	`network::Server`

With one `run()` thread the strand is redundant today. Don't add a second one: `TcpClient`'s connect handlers aren't on a strand, and `Connection::start()` starts its first read off the strand.

Locks around asio: how it fails

Never

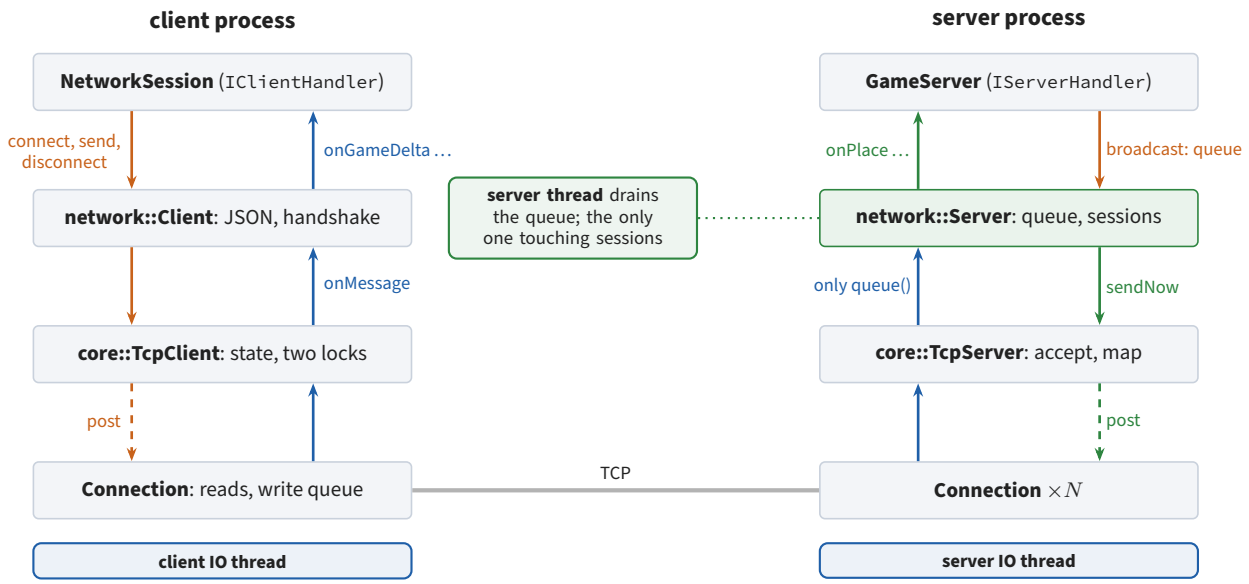
- **join the IO thread from a handler.** `stop()` or a destructor called in a callback joins the thread it runs on: `join()` throws and the program terminates. Asserts in `TcpServer::stop`, `Server::stop` and `~TcpClient` catch it.
- **wait in a handler** for a thread that waits for that handler. `TcpClient::connect()` and `disconnect()` wait for a running callback.
- **hold a lock while calling out**, if the callee may take it too. A `std::mutex` locked twice on one thread hangs; that is why `TcpClient`'s callback lock is recursive.
- **block the IO thread.** Every connection of that context stalls with it.

Who keeps it alive while a handler waits?

object	kept alive by
`Connection`	`self = shared_from_this()`, captured in every handler
read header, payload; write header	`make_shared`, captured in the handler
write payload	`m_writeQueue.front()`, popped only after its write. `deque::push_back` never moves elements; a `vector` would!
resolver, socket while connecting	`shared_ptr`s captured in the handlers (`TcpClient::connect`)
`this` in TcpServer, TcpClient handlers	the destructor stops and joins the IO thread before any member dies
`network::Client` members	`m_client` is declared last, so it is destroyed first: no callback reaches a dead member

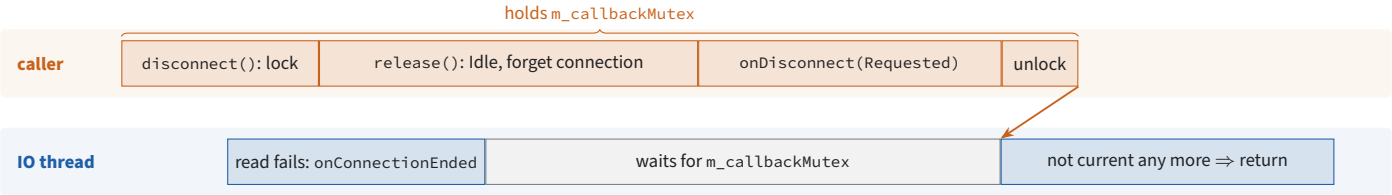
Threads in Tengen

arrow colour = the thread that runs the call



Calls go down, callbacks come up. Dashed = `post`: the caller only queues, the **IO thread** does the I/O. The client has no thread of its own, so `NetworkSession`'s handler runs *on the IO thread*: it must be quick and lock its own state. The server leaves the IO thread at once, so a slow game never stalls the sockets. `GameServer` hands moves to the game's own thread, which broadcasts start, deltas and end; only chat is broadcast from the **server thread**.

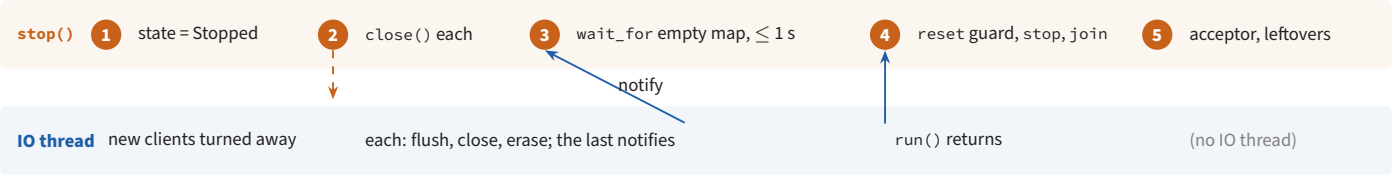
TcpClient: two locks, one report per connection



- `m_callbackMutex` is held around *every* callback: they run one at a time, and `disconnect()` returns only once no callback of that connection can run any more.
- It is **recursive**: a callback may call `disconnect()` on the same thread. `network::Client::close()` does, on a reject or a bad message.
- `m_mutex` guards the state and is **never** held during a callback, so `send()` and `isConnected()` never wait for one.
- Lock order: callback lock first, then state lock.

Shutting down in order

`tcpServer.cpp` · `server.cpp`



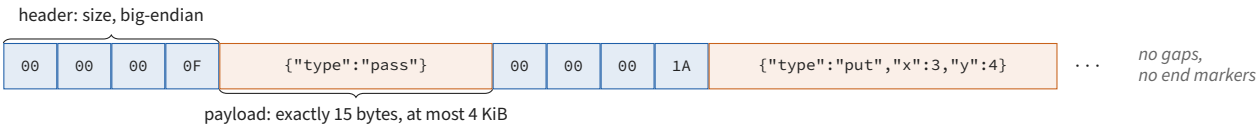
- 3 `wait_for` *releases* `m_connectionsMutex` while it waits, so the IO thread can erase and notify.
- 5 only after the join: the acceptor and the sockets of slow clients have no other thread using them any more (rule 5).
- `network::Server::stop()` first pushes a `Shutdown` event under its state lock and joins the server thread. The queue is FIFO, so every `send`/`broadcast` before it reaches the connections, and nothing can be queued after it. Then `TcpServer::stop()` flushes and closes.

Worth knowing when reviewing

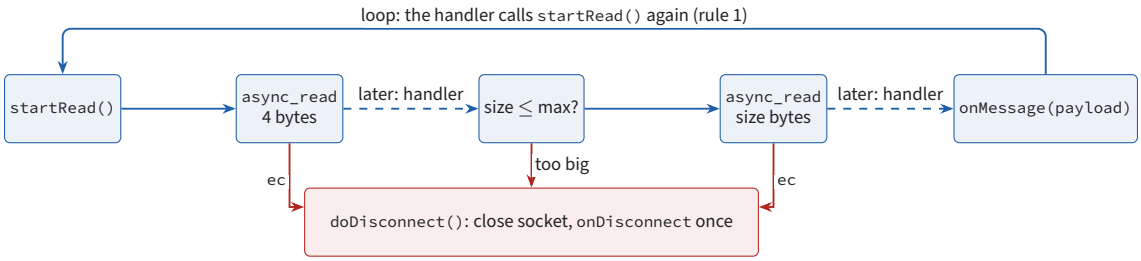
`doAccept` calls `start()` → `onConnect` while holding `m_connectionsMutex`. Fine today, since `network::Server`'s callback only queues. A callback that calls `TcpServer::send()` or `disconnect()` there would lock it a second time and hang.

Example 1 · Connection: framing and the read loop

src/net/core/connection.cpp



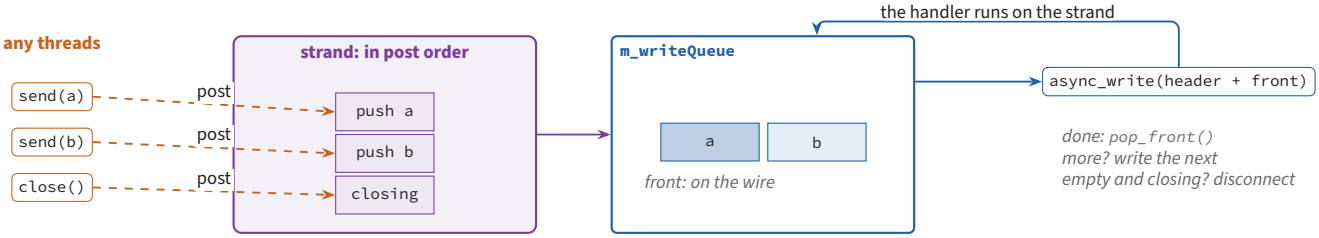
TCP keeps the order of bytes, not the boundaries of messages: one send can arrive in pieces, two can arrive glued together. So each message carries its length, and the reader asks `async_read` for exactly 4 bytes, then for exactly *size* bytes. A size above `MAX_PAYLOAD_BYTES` means the stream is lost: there is no way to find the next boundary, so the connection closes.




```
auto header = std::make_shared<BasicMessageHeader>(); // rule 4
auto self   = shared_from_this();                     // rule 4
asio::async_read(m_socket,
    asio::buffer(header.get(), sizeof(BasicMessageHeader)),
    asio::bind_executor(m_strand,
        [self, header](asio::error_code ec, std::size_t) {
    if (ec || !self->m_running) {  // rule 7: first thing
        self->doDisconnect();      // closed by us or the peer
        return;
    }
    // ... size check, read payload, onMessage, startRead()
}));
```

- `header` and `self` ride along in the lambda, so they live exactly as long as the read.
- A local `BasicMessageHeader h;` with `buffer(&h, 4)` would be written to after `startRead()` returned: a dangling buffer.
- `doDisconnect()` starts with `m_running.exchange(false)`: whoever ends the connection first reports it, the others return quietly.

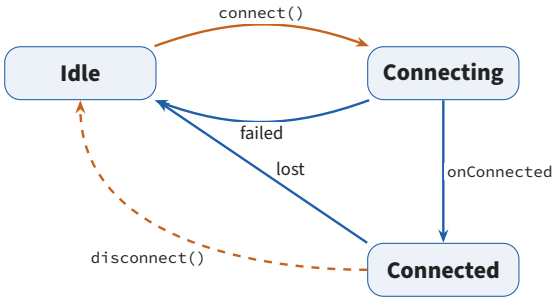
Example 2 · Connection: the write queue



- **Why post?** The queue and its flags belong to the strand. `send()` moves the message into the lambda and returns.
- **Why a queue?** Rule 6: the next `async_write` may start only in the handler of the last (`m_writeInProgress`).
- **Why is `close()` posted too?** It lands *behind* the sends posted before it, so they still go out. Sends posted after it see `m_closing` and are dropped.

Example 3 · TcpClient: handlers that come too late

src/net/core/tcpClient.cpp



The **caller** starts and ends, the **IO thread** reports what happened. `disconnect()` works from Connecting too. Every way back to Idle calls `onDisconnect` exactly once (destroying the client calls none); after `disconnect()` it runs on the caller's thread, before `disconnect()` returns.

The race. `disconnect(); connect();` quickly: the first attempt's handler still runs, with `operation_aborted` or even success (rule 7), and must not touch the second attempt. Each handler captured *its* socket, so it compares:

```
if (socket != m_socket) return; // let go of already
```

No ABA: while the handler holds the old socket, no new socket can get its address.

Cancelling from the caller's thread (rule 5):

```
asio::post(m_ioContext,
    [resolver = std::exchange(m_resolver, nullptr),
     socket   = std::exchange(m_socket, nullptr)] {
  resolver->cancel();  // on the IO thread
  asio::error_code ec;
  socket->close(ec);   // handlers: operation_aborted
});
```

`exchange` does two jobs: it hands the objects to the IO thread and empties the members, which makes the attempt stale.